スポットインスタンスの
スケジューリング戦略（簡易版）

那須 渚

2026 年 10 月 12 日

1　やりたいこと

これまでの研究で，$F_{\mathrm{comp}}(d)$（期限 d までインスタンスが中断されない確率）を測定し，**夜間が最も中断されにくい**ことがわかった．

しかし，**測っただけで，それを使って何かを決める仕組みがない．**

そこで次は，$F_{\mathrm{comp}}(d)$ を使って

ジョブをどのリージョンで・どのタイプで・いつ実行するか

を自動で決める戦略をつくり，GitHub Actions で効果を確かめる．

2　考え方

中断されるとジョブは最初からやり直しになる．つまり**中断されやすい時間帯を選ぶと，やり直しの分だけ余計にお金がかかる．**

この「結局いくらかかるか」を計算できれば，一番安い条件を選べる．計算は 3 段階で進む．

2.1　(1) 平均何回やれば終わるか

p を 1 回で成功する確率とする（$= F_{\mathrm{comp}}$ の実測値）．成功するまで繰り返すときの平均回数は，

$$\mathbb{E}[\text{試行回数}] = \frac{1}{p} \tag{1}$$

「10 回中 7 回成功するなら，1 回成功させるのに 10/7 回必要」という意味である．

表 1　実測値の例（N. Virginia / c6a.large / $d = 60$ 分）

時間帯	p	平均試行回数 $1/p$
Night	0.98	1.02 回
Morning	0.69	1.45 回

2.2 (2) 合計で何分ぶん課金されるか

課金される時間は 2 つに分かれる．

- **成功した 1 回** $\to d$ 分ぶん
- **失敗した回数**（平均 $\frac{1}{p}-1$ 回）$\to$ 中断されるまでの時間 $\bar{u}$ ぶん

$\bar{u}$ は「中断されたジョブが平均何分走ってから落ちたか」で，観測データ（`uptime_s`）から直接わかる．

$$\mathbb{E}[\text{課金時間}] \;=\; d \;+\; \left(\frac{1}{p}-1\right)\times \bar{u} \tag{2}$$

■例（$d=60$ 分，$\bar{u}\approx 12$ 分）．

$$\text{Night } (p=0.98): \quad 60+0.02\times 12 = 60.2 \text{ 分} \tag{3}$$

$$\text{Morning } (p=0.69): \quad 60+0.45\times 12 = 65.4 \text{ 分} \tag{4}$$

Morning は Night より **5.2 分ぶん余計に**課金される．これが時間帯を選ぶ意味である．

2.3 (3) 金額に直す

スポット価格 P_{spot} は 1 時間あたりなので，60 で割って分に合わせる．

$$C = \frac{P_{\text{spot}}}{60}\times \mathbb{E}[\text{課金時間}] \tag{5}$$

$P_{\text{spot}}=0.0295$ USD/h のとき，Night は 0.0296 USD，Morning は 0.0322 USD となる．

3 戦略：一番安い条件を選ぶ

ここまでで「リージョン r・タイプ τ・起動時刻 t を決めればコストが計算できる」状態になった．あとは**全部試して一番安いものを選ぶ**．

候補は 4 リージョン $\times$ 3 タイプ $\times$ 24 時刻 $=288$ 通り しかないので，全列挙でよい．

ただし，ただ安いだけだと「来週の深夜に実行します」という答えが出てしまう．そこで次の 2 つの条件をつける．

表 2　戦略が守る条件

条件	意味
締切	t は t_0 から $t_0+(D_j-d_j)$ までの範囲で選ぶ
信頼性	成功確率が $p_{\min}$ 未満の条件は選ばない

■締切条件（slack）. D_j は「いつまでに終わればよいか」，d_j は実行時間なので，**遅らせてよい上限**は $D_j - d_j$ になる．これを **slack** と呼ぶ．

例：10 分のジョブで「6 時間以内でよい」なら，約 5 時間 50 分まで待てる．

slack $= 0$ なら今すぐ実行するしかなく，**選べるのはリージョンとタイプだけ**になる．これは既発表の “region selection can substitute for time-shifting” に対応する．

■信頼性条件. 期待値はあくまで平均なので，「成功率 30% でも，何度もやり直せば平均は間に合う」という計算結果が出てしまうことがある．しかし 3 回に 2 回失敗する CI は実務で使えない．そこで $p_{\min}$（例：0.8）という下限を直接設ける．

4 何と比べるか

戦略の効果を示すには，比較相手が要る．次の 5 つを比べる．

表 3 比較する戦略

戦略	やること	役割
S0: Naive	今すぐ・固定の条件で実行	GitHub Actions の素の挙動
S1: Cheapest	一番安い価格を選ぶ	価格だけ見る実務的手法
S2: Region-only	今すぐ・リージョンだけ選ぶ	時間シフトできない場合
S3: Proposed	リージョン・タイプ・時刻を全部選ぶ	**提案手法**
S4: Oracle	結果を知った上での最良	到達可能な上限

■S2 を入れる理由. S2 と S3 を比べると，**時間をずらした効果**と**リージョンを変えた効果**を分けて示せる．査読でほぼ必ず聞かれる点なので，最初から分けておく．

5 注意：少ないデータを信じすぎない

F_{comp} は 30–42 件から推定した値なので，たまたま高く出たセルがある．そのまま「一番高いセル」を選ぶと，運が良かっただけの条件を選んでしまう（winner’s curse）．

対策は，推定値そのものではなく 95% **信頼区間の下端**（Wilson の下側信頼限界）を使うこと．

- サンプルが少ないほど，この値は大きく下がる
- つまり**データの少ない条件を自動的に避けてくれる**

計算は `statsmodels` の `proportion_confint(k, n, method='wilson')` で一行である．通常版（S3）と保守版（S3-LCB）の両方を比べれば，それ自体が結果になる．

6 実験のやり方

6.1 GitHub Actions の構成

表 4 ワークフローの 4 ステップ

ジョブ	実行場所	やること
decide	GitHub hosted	戦略に従って (r, τ, t) を決める
provision	GitHub hosted	決めた時刻まで待ち，スポットを起動して runner を登録
workload	スポット上	実際の CI ジョブを走らせる
collect	GitHub hosted	結果（成功・中断・コスト・時間）を記録

中断の検知は, 既存の `start_instances.py` と同じ方法（インスタンス内から `spot/instance-action` を監視）が使える．検知したら失敗として記録し，provision からやり直す．

■注意点． runner は必ず `--ephemeral` で登録すること．そうしないと中断後に古い runner が残り，次の試行が汚れる．

6.2 振るパラメータ

表 5 実験パラメータ

パラメータ	水準	備考
d_j（実行時間）	$5, 10, 30, 60$ 分	既発表と同じ
slack	$0, 1, 3, 6, 12$ 時間	0 は時間シフト不可
$p_{\min}$	$0, 0.8, 0.9$	0 は制約なし
戦略	S0–S4, S3-LCB	

6.3 測るもの

- 実際にかかったコスト（USD）
- 完了までの時間
- やり直した回数
- 締切に間に合った割合
- **式 (2) の予測と実測のズレ**

最後の項目が重要で，これは**式が現実を説明できているか**の検証になる．

7　まとめ

表 6　使う式は 3 つだけ

式	意味
$\dfrac{1}{p}$	成功するまでの平均回数
$d + \left(\dfrac{1}{p} - 1\right)\bar{u}$	合計の課金時間（分）
$\dfrac{P_{\text{spot}}}{60} \times$ (上式)	金額に換算

必要なデータは次の 3 つだけで，**すべて手元にある．**

- $p = F_{\text{comp}}$（計算済み）
- $\bar{u}$ = 中断時の `uptime_s`（データにある）
- P_{spot} = スポット価格（実測済み）

■次にやること．　まず既存の CSV から上の 3 つの表をつくり，S0–S4 をシミュレーションで比較する `strategy.py` を書く．

GitHub Actions の実機実験はその後でよい．シミュレーションで**差が出る条件を先に見つけてから**実機に移るほうが，お金も時間も節約できる．

詳細版（導出・最適化問題としての定式化・実装コード）は `future-main.pdf` を参照．